



Universidad Experimental Nacional de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: INGENIERÍA DE SOFTWARE II

UNIDAD V

GESTIÓN DE CONFIGURACIÓN, VERSIONAMIENTO Y DEVOPS

Profesor:

Marquez Felix

Estudiantes:

Alondra Moreno CI: 30.331.870

Samuel Cortez CI: 27.158.842

Carlos Hernández CI: 27.975.753

Eduardo Quintero CI: 32.004.965

Ciudad Guayana, Septiembre 2026

Introduccion

La presente resolución técnica aborda las asignaciones de la Unidad 5 de la asignatura Ingeniería de Software II, centrada en la Gestión de Configuración, Versionamiento y DevOps. Históricamente, la Gestión de Configuración de Software (SCM) se limitaba al control de versiones del código y a procesos burocráticos centralizados. Sin embargo, este informe documenta la transición práctica hacia el paradigma contemporáneo de *Everything as Code* (Todo como Código), donde la infraestructura, los entornos y las reglas de integración se gestionan con el mismo rigor que el código fuente aplicativo. A través de cuatro secciones analíticas y prácticas, el equipo aborda la resolución de conflictos mediante modelos de ramificación ágiles (GitHub Flow y Trunk-Based Development), la automatización de la calidad mediante Integración Continua (CI) y *Quality Gates*, y la inmutabilidad de los despliegues utilizando contenedores Docker. El objetivo es demostrar cómo estas metodologías modernas eliminan la deriva de configuración y aceleran la entrega de valor sin comprometer la seguridad ni la integridad del producto de software.

PREGUNTA 1: FLUJO DE TRABAJO EN GIT, PULL REQUESTS Y DOCKER (CASO SAAS DE FACTURACIÓN)

1. Diagrama de Flujo y Secuencia Git (GitHub Flow)

Parte A: Desarrollo de la Funcionalidad (Rama feature/pdf-export)

En el flujo GitHub Flow, la rama 'main' siempre representa el código listo para producción. Toda nueva funcionalidad debe aislarse en una rama descriptiva de corta duración.

- **1. Sincronización Inicial:** Asegurar que la rama main local esté actualizada con el servidor remoto:

`git checkout main`

`git pull origin main`
- **2. Creación de la Rama Feature:** Crear y posicionarse en la nueva rama para el módulo de facturación:

`git checkout -b feature/pdf-export`
- **3. Trabajo y Commits Atómicos:** Modificar archivos, agregar librerías de generación de PDF y registrar los cambios de forma atómica:

`git add src/services/pdfService.js package.json`

`git commit -m "feat(billing): implementar servicio de generación de facturas en PDF"`
- **4. Publicación Remota:** Publicar la rama en el repositorio remoto de GitHub/GitLab:

`git push -u origin feature/pdf-export`
- **5. Apertura del Pull Request:** Crear el Pull Request (PR) apuntando hacia 'main', solicitar Code Review a los pares y permitir la ejecución automática de los Quality Gates de CI.

- **6. Merge e Higiene de Ramas:** Una vez aprobado el PR y superadas las pruebas de CI, realizar el Merge a 'main' y eliminar la rama feature remota y local para mantener el repositorio limpio:

git checkout main

git pull origin main

git branch -d feature/pdf-export

git push origin --delete feature/pdf-export

Parte B: Gestión de Hotfix de Producción durante el Desarrollo

Escenario: Mientras el desarrollador trabaja en 'feature/pdf-export', se detecta un bug crítico de inicio de sesión en producción (rama 'main').

- **1. Aislamiento del Trabajo Actual:** Guardar temporalmente los cambios en progreso de la función PDF sin hacer commit innecesario:

git stash save "WIP: exportación PDF antes de atender hotfix de login"

- **2. Cambio a Main:** Regresar a la rama principal y sincronizar:

git checkout main

git pull origin main

- **3. Creación de Rama Hotfix:** Crear una rama de corrección rápida a partir de main:

git checkout -b hotfix/login-authentication-crash

- **4. Aplicación de la Solución:** Corregir el fallo en el módulo de autenticación, probar e impactar el commit:

git add src/auth/loginController.js

git commit -m "fix(auth): corregir fallo de autenticación JWT en inicio de sesión"

- **5. Despliegue Urgente:** Subir la rama, abrir PR de emergencia (Fast-track Review) y fusionar a main tras validar en CI:

```
git push origin hotfix/login-authentication-crash
```

```
# Tras aprobación y merge en GitHub/GitLab:
```

```
git checkout main
```

```
git pull origin main
```

```
git branch -d hotfix/login-authentication-crash
```

- **6. Retorno al Desarrollo de Feature:** Reanudar el trabajo en la rama de facturación PDF e integrar la corrección del hotfix mediante rebase para evitar conflictos futuros:

```
git checkout feature/pdf-export
```

```
git rebase main
```

```
git stash pop
```

2. Plantilla Profesional de Pull Request (PR)

A continuación se presenta la plantilla en formato Markdown diseñada para estandarizar las revisiones de código en la funcionalidad de exportación a PDF:

Descripción del Cambio

Se implementa el servicio de exportación de facturas en formato PDF (`pdfService.js`) utilizando la librería `pdfkit` v0.13.

- Permite descargar facturas individuales desde el panel de clientes.

- Incluye marca de agua, desglose de impuestos (IVA) y firma digital de la empresa.

Tipo de Cambio

- [] Corrección de error (Bug fix)
- [x] Nueva funcionalidad (New feature)
- [] Cambio rompedor (Breaking change)
- [] Refactorización / Mejora de rendimiento

Impacto en el Sistema y Riesgos

- **Componentes Afectados:** Módulo de Facturación (`/api/v1/invoices`), generación de reportes y almacenamiento temporal en disco (`/tmp`).
- **Riesgo Potencial:** Alto consumo de memoria RAM si se solicitan más de 50 descargas simultáneas en entornos de escasos recursos.
- **Mitigación:** Se implementó generación por Streams de Node.js en lugar de almacenamiento en búfer de memoria.

Checklist de Revisión Obligatorio

- [x] **Pruebas Unitarias:** Pruebas de generación de PDF ejecutadas exitosamente (Cobertura > 85%).
- [x] **Seguridad en Librerías:** Escaneo de dependencias realizado con `npm audit` / Snyk (0 vulnerabilidades conocidas en `pdfkit`).
- [x] **Sanitización de Datos:** Validación e higienización de entradas de usuario para evitar inyección de encabezados en el PDF.
- [x] **Estándares de Código:** Código verificado mediante ESLint sin advertencias o errores de sintaxis.

- [x] ****Documentación:**** Swagger / OpenAPI actualizado con el nuevo endpoint ``/api/v1/invoices/:id/pdf``.

3. Automatización en CI (Quality Gates)

Las Puertas de Calidad (Quality Gates) son controles automatizados e infranqueables en el pipeline de Integración Continua (CI) que bloquean la fusión de código defectuoso o vulnerable:

- **1. Análisis Estático de Código y Seguridad de Dependencias (SAST):** Un linter estático (ESLint) verifica la conformidad del código con las reglas de estilo de la organización, mientras herramientas SAST (Static Application Security Testing como SonarQube o Snyk) analizan el código fuente y las dependencias (ej. `pdfkit`) en busca de fallos de seguridad (OWASP Top 10) y licencias incompatibles antes de cualquier compilación.
- **2. Ejecución Automatizada de Pruebas y Cobertura de Código:** El pipeline ejecuta la suite completa de pruebas unitarias y de integración en un entorno aislado. El Quality Gate exige que el 100% de los tests pasen exitosamente y que la cobertura de código (Code Coverage) nueva no sea inferior al 80%. Si un test falla o la cobertura disminuye, el PR queda automáticamente bloqueado.
- **3. Construcción del Artefacto y Análisis de Vulnerabilidades en Imágenes (Container Security Scan):** El pipeline construye el contenedor Docker de la aplicación y ejecuta un escáner de imágenes de contenedores (como Trivy o Clair). Se verifica que la imagen base no contenga vulnerabilidades críticas (CVEs) del sistema operativo subyacente y que el binario o empaquetado final compile limpiamente sin advertencias críticas.

Ejercicio 2

1. Dockerización de la Aplicación Node.js (Dockerfile)

A continuación se presenta el `Dockerfile` optimizado utilizando la técnica Multi-Stage Build para entornos de producción:

```
# =====
```

```
# ETAPA 1: Construcción y Dependencias (Build)
```

```
# =====
```

```
FROM node:18-alpine AS builder
```

```
# Establecer el directorio de trabajo dentro del contenedor
```

```
WORKDIR /app
```

```
# Copiar archivos de definición de paquetes primero (para aprovechar el almacenamiento  
en caché de capas)
```

```
COPY package*.json ./
```

```
# Instalar todas las dependencias (incluidas las de desarrollo para pruebas)
```

```
RUN npm ci
```

```
# Copiar el resto del código fuente del proyecto
```

```
COPY . .
```

```
# =====
```

ETAPA 2: Entorno Ligero de Producción

=====

FROM node:18-alpine AS runner

Establecer variable de entorno a producción

ENV NODE_ENV=production

WORKDIR /app

Crear un usuario no raíz por razones de seguridad (Principio de Menor Privilegio)

RUN addgroup -g 1001 -S nodejs && adduser -S nodejs -u 1001

Copiar solo las dependencias de producción y archivos necesarios desde la etapa de construcción

COPY package*.json ./

RUN npm ci --only=production && npm cache clean --force

COPY --from=builder --chown=nodejs:nodejs /app/server.js ./

COPY --from=builder --chown=nodejs:nodejs /app/src ./src

Cambiar al usuario no privilegiado

USER nodejs

Exponer el puerto en el que escucha la aplicación de facturación

EXPOSE 8080

Comando para ejecutar la aplicación

```
CMD ["node", "server.js"]
```

Explicación Técnica: Resolución de la Deriva de Configuración y 'Funciona en mi Máquina':

El problema clásico 'funciona en mi máquina' ocurre porque cada desarrollador posee diferentes versiones de Node.js, librerías del sistema operativo (ej. dependencias nativas de PDF en Linux vs macOS) o variables de entorno no documentadas. Docker encapsula la aplicación junto con su sistema operativo exacto, versión del runtime Node.js, código y dependencias en un sistema de archivos inmutable. Esto elimina la 'deriva de configuración' (Configuration Drift) asegurando que el contenedor ejecutado en la máquina del desarrollador sea idéntico en un 100% al que se ejecuta en el servidor de homologación y producción en la nube.

2. Reflexión: Trazabilidad y Auditoría en IaC vs. Registros Manuales

Gestionar la infraestructura mediante código (Infrastructure as Code - IaC con Terraform, CloudFormation o Docker Compose) almacenado en Git proporciona ventajas abismales frente a las bitácoras o logs manuales de un administrador de sistemas:

- **1. Trazabilidad Inmutable:** Cada cambio en la infraestructura (apertura de puertos, escalado de servidores, bases de datos) queda registrado en el historial de commits de Git con autor, fecha, hora exacta y la justificación del cambio en el mensaje del commit.
- **2. Control de Cambios Auditable (Peer Review):** Cualquier modificación a la infraestructura debe pasar por un Pull Request donde otros ingenieros revisan el código de infraestructura antes de aplicarlo, evitando errores humanos accidentales en consolas web de AWS/Azure.

- **3. Reversibilidad (Rollback):** Si una actualización de infraestructura falla en producción, es posible ejecutar un `git revert` y desplegar el estado anterior en minutos, eliminando el riesgo de configuraciones manuales olvidadas en notas de texto.

PREGUNTA 2: ANÁLISIS COMPARATIVO DE MODELOS DE RAMIFICACIÓN (BRANCHING MODELS)

1. Cuadro Comparativo: GitFlow vs. Trunk-Based Development (TBD)

Criterio	GitFlow	Trunk-Based Development (TBD)
Duración de Ramas	Larga duración (semanas o meses para ramas feature, release y develop).	Muy corta duración (pocas horas o máximo 1-2 días).
Rama Principal	Dos ramas de larga duración: main (producción) y develop (integración).	Una única rama central activa (trunk o main).
Frecuencia de Integración	Infrecuente. La integración ocurre al final de ciclo de desarrollo extenso.	Continua y múltiple veces al día directamente en el trunk.
Riesgo de Conflictos (Merge Hell)	Muy Alto. Las divergencias de código acumuladas provocan fusiones complejas y dolorosas.	Mínimo. Al ser commits pequeños y frecuentes, los conflictos se detectan e integran de inmediato.
Aptitud para CI/CD	Baja / Compleja. Diseñado para ciclos de liberación tradicionales estructurados.	Alta / Nativa. Diseñado expresamente para Despliegue Continuo (Continuous Deployment).

Mecanismo de Despliegue	Basado en congelamiento de código (Code Freeze) y ramas de release.	Basado en pequeñas iteraciones ocultas tras Feature Flags.
-------------------------	---	--

2. Argumentación basada en 'Accelerate' (Forsgren, Humble & Kim)

En el estudio empírico seminal 'Accelerate: The Science of Lean Software and DevOps', los autores demostraron que las organizaciones de alto desempeño (High Performers) evitan sistemáticamente los modelos de ramas de larga duración como GitFlow a favor de Trunk-Based Development por las siguientes razones analíticas:

- **Reducción del Lead Time for Changes:** Los autores descubrieron que el indicador clave de rendimiento en ingeniería de software es el tiempo que tarda un cambio desde que se escribe hasta que llega a producción. GitFlow destruye este indicador debido a las esperas en 'develop', los períodos de 'congelamiento de código' y la burocracia de las ramas de release.
- **Eliminación del 'Merge Hell' e Integración Tardía:** Cuando los desarrolladores mantienen ramas aisladas por semanas, se genera el infierno de fusión ('Merge Hell'). Las pruebas de integración se postergan hasta el final del ciclo, ocultando deuda técnica y errores de arquitectura. TBD fuerza la integración continua diaria, convirtiendo el dolor de la integración en pequeñas verificaciones triviales.
- **Frecuencia de Despliegue y Estabilidad del Sistema:** El estudio demuestra que despliegues más pequeños y frecuentes reducen drásticamente la tasa de fallas en cambios (Change Failure Rate) y permiten una recuperación rápida (MTTR) en caso de incidencias.

3. Prácticas Obligatorias de SCM e Ingeniería para TBD

Para implementar Trunk-Based Development de forma segura sin comprometer el código en producción, se requieren tres pilares técnicos indispensables:

- **1. Interruptores de Funcionalidad (Feature Flags / Feature Toggles):** Mecanismos condicionales en el código que permiten integrar funciones incompletas en la rama principal desactivadas para los usuarios finales. Esto permite desplegar código continuamente a producción sin liberar la funcionalidad hasta que esté totalmente terminada y probada.
- **2. Integración Continua Automatizada de Alta Velocidad (Fast Feedback CI):** Como docenas de commits se fusionan diariamente en 'main', es imposible realizar pruebas manuales. Se requiere un pipeline de CI automatizado que ejecute pruebas unitarias, de integración y seguridad en menos de 10 minutos por cada commit.
- **3. Integración en Pequeños Batches (Small Batch Size & Micro-Commits):** Los desarrolladores deben desacoplar los commits grandes en pequeños cambios atómicos que puedan revisarse e integrarse en horas mediante Pair Programming o revisiones de código livianas e inmediatas.

PREGUNTA 3: EVOLUCIÓN DEL CONTROL DE CAMBIOS

1. Crítica a la Junta de Control de Cambios (CCB) Tradicional

Afirmación a discutir: 'La Junta de Control de Cambios (CCB) tradicional es un cuello de botella que viola los principios de la Entrega Continua'.

Postura: TOTALMENTE DE ACUERDO. Análisis Técnico y Discusión:

La Junta de Control de Cambios (Change Control Board - CCB) nació en la era de la gestión de proyectos de ITIL tradicional, donde un comité de gerentes se reunía semanalmente o mensualmente para revisar formularios impresos o en Word antes de autorizar el despliegue de un software. En el paradigma moderno de Entrega Continua (Continuous Delivery), la CCB tradicional representa un obstáculo destructivo por las siguientes razones:

- **Cuello de Botella Temporal:** Retrasa el despliegue de software por días o semanas mientras se espera la reunión del comité, destruyendo la agilidad del negocio.
- **Aumento del Tamaño de Lote (Batch Size):** Como la CCB aprueba cambios en bloques espaciados, los equipos acumulan cientos de cambios en un solo despliegue masivo. Esto aumenta exponencialmente el riesgo de fallas críticas y dificulta identificar qué cambio específico causó un error.
- **Falsa Sensación de Seguridad:** Los miembros del comité raramente leen el código fuente ni comprenden la arquitectura técnica interna, basando su decisión en impresiones subjetivas o garantías documentales vacías que no garantizan la calidad real del software.

2. Control de Cambios Automatizado: Peer Review + CI Pipeline

La ingeniería de software moderna ha reemplazado la burocracia de la CCB por un mecanismo de control de cambios continuo, auditable y automatizado:

- **1. Revisión de Pares (Peer Review Inmediato):** Al abrir un Pull Request, al menos uno o dos ingenieros de software pares (Peer Reviewers) analizan la lógica, arquitectura y legibilidad del código. Este control entre iguales es técnicamente superior a cualquier revisión gerencial.

- **2. Validación Infranqueable mediante CI (Quality Gates):** En lugar de que un comité asuma que las pruebas se hicieron, el pipeline de CI prueba el software automáticamente en segundos. Valida que el código compile, las pruebas pasen, las vulnerabilidades sean cero y los estándares de seguridad se cumplan.
- **3. Trazabilidad y Auditoría Digital Inmutable:** Cada PR aprobado y fusionado en Git deja un registro digital inalterable con la firma del autor, la firma del revisor, el log de ejecución de las pruebas automáticas de CI y la fecha/hora exacta. Esto cumple de forma nativa con normativas de auditoría internacional (ISO 27001, SOC2, HIPAA) sin llenar formularios manuales.

PREGUNTA 4: ARTEFACTOS Y AUDITORÍA EN DEVOPS

1. Evolución del 'Plan de Gestión de Configuraciones'

Antiguamente, el Plan de Gestión de Configuraciones (CMP) era un documento de Microsoft Word estático de cientos de páginas que intentaba detallar qué servidores existían, qué programas debían instalarse y qué personas tenían permisos. Estos documentos quedaban desactualizados el mismo día en que eran aprobados.

En el paradigma DevOps, la documentación estática ha sido reemplazada por el principio de 'Todo como Código' (Everything as Code). Los planes de configuración hoy son archivos de texto plano ejecutables almacenados en la raíz del propio repositorio de Git. De esta forma, la configuración no se 'lee' en un manual, sino que se 'ejecuta' mediante scripts automatizados, asegurando que la documentación y la realidad técnica de producción coincidan exactamente en todo momento.

2. Análisis de 4 Archivos de Configuración Modernos

- **1. .gitignore:** Archivo de texto plano que especifica los patrones de archivos, directorios y artefactos temporales que Git debe ignorar deliberadamente. Previene la filtración accidental de credenciales sensibles (`.env`), paquetes de terceros pesados (`node_modules/`) y archivos temporales de compilación en el repositorio remoto, garantizando la seguridad e higiene del código base.
- **2. Dockerfile:** Receta ejecutable que define la infraestructura y el entorno de tiempo de ejecución exacto de una aplicación mediante contenedores. Especifica el sistema operativo base, variables de entorno, dependencias de compilación y comandos de arranque. Garantiza que la aplicación se ejecute en un entorno inmutable e idéntico en cualquier servidor.
- **3. .github/workflows/ci.yml:** Archivo de definición en formato YAML que especifica el pipeline automatizado de Integración Continua y Despliegue Continuo (CI/CD). Define las etapas de compilación, ejecución de Quality Gates, pruebas automatizadas, escaneo de seguridad y despliegue en la nube cuando se eventos de Git (push, PR). Garantiza la calidad e integridad automática del software.
- **4. docker-compose.yml:** Archivo de orquestación en formato YAML utilizado para definir y ejecutar aplicaciones compuestas por múltiples contenedores Docker (por ejemplo, servidor web Node.js + base de datos PostgreSQL + caché Redis). Permite levantar toda la arquitectura de microservicios distribuida en un solo comando (`docker-compose up`), garantizando la reproducibilidad exacta de entornos complejos de prueba y desarrollo.

Conclusion

El desarrollo de esta unidad demuestra que la evolución de la Gestión de Configuración (SCM) hacia DevOps sustituye los lentos controles documentales por mecanismos técnicos automatizados y deterministas. La transición de modelos rígidos hacia *Trunk-Based Development* y *GitHub Flow* reduce drásticamente el tiempo de entrega y minimiza los costosos conflictos de integración masiva (*Merge Hell*). Paralelamente, se comprobó que la tradicional Junta de Control de Cambios (CCB) representa un cuello de botella en escenarios de entrega continua, siendo superada ampliamente por la validación técnica mediante *Pull Requests* y pipelines de Integración Continua (CI) que auditan la seguridad y la calidad del código en tiempo real. Por último, el encapsulamiento de dependencias en contenedores Docker y la definición de la Infraestructura como Código (IaC) garantizan la inmutabilidad y la perfecta trazabilidad de cada entorno de ejecución. En conjunto, la adopción estructurada de estas herramientas arquitectónicas y culturales resulta indispensable para operar ciclos de ingeniería de software rápidos, resilientes y altamente auditables.

Bibliografía

- Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley. (Capítulo sobre gestión de configuraciones y despliegue).
- Driessen, V. (2010). A successful Git branching model. nvie.com.
- Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.
- Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
- IEEE Computer Society. (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK V3/V4). IEEE.
- Prokic, S. (2021). Infrastructure as Code: Dynamic Infrastructure with Terraform. Packt Publishing.